

Structures

Recall that we finished our last class speaking about inductive types, and among those we mentioned

```
structure Iff (P Q : Prop) : Prop
  number of parameters: 2
  fields:
    Iff.mp : P → Q
    Iff.mpr : Q → P
  constructor: Iff.intro {P Q : Prop} (mp : P → Q) (mpr : Q → P) : P ↔ Q
```

Why did `#print Iff` begin with `structure` rather than with `inductive`?

Because it is a *structure* (with two fields):

Definition

A structure is an inductive type with a unique constructor.

Indeed, among inductive types (*i. e.* all types...), some are remarkably useful for formalising mathematical objects: those that *bundle* objects and properties together. So, we give them a different name.

As an example, let's see what a Monoid is:

```
structure Monoid (M : Type*) where
  | mul : M → M → M           -- denoted *
  | one : M                     -- denoted 1
  | mul_assoc (a b c : M) : a * b * c = a * (b * c)
  | one_mul (a : M) : 1 * a = a
  | mul_one (a : M) : a * 1 = a
```

- Two of these fields are terms in types of kind `Type *`;
- three of them are terms in types of kind `Prop`;
- we often call a structure having constructor fields both in `Type *` and in `Prop` a *mixin*.

So,

- a *monoid structure* on `M` is a collection `{*, 1, mul_assoc, one_mul, mul_one}`
- a term of a monoid is just a term of it! The monoid is a type, so it comes with its terms even if it has more structure, which is encoded in a term `str : Monoid M`.

⌘

Algebraic structures

Let's begin with some *painful* example: ⌘

They show that algebraic manipulations "from first principles" are doable, but certainly the *wrong* way to go.

Groups

In this section we're going to see

1. A bad definition of (the right notion of) group;
2. A complicated one of the same notion, adapted for formalisation — and its advantages;
3. How to benefit from Mathlib, a huge library where all this has already been defined.



The problems with `WrongGroup` and `WrongSemigroup` are (among others...)

- We're carrying around `mul`, `one` and `inv`, together with the type: `α.one` or `X.mul` or `G.inv`...
- Math is full of hierarchies, and these are not respected (the associativity example...): but we don't want to re-prove a theorem on additive commutative groups for rings, then for commutative rings, then for integral domains, then for fields...
- Although we can create

```
def Nplus : WrongSemigroup where
  carrier := ℕ
  mul := (· + ·) -- or fun x y => x + y
  mul_assoc := add_assoc
```

it is unhandy to connect `Nplus` with `ℕ` as types.

Extends

The right approach relies on the idea of *extending* structures.

Suppose we've already defined a structure `PoorStructure` with fields `firstfield`, ..., `nth_field` and we want a new *richer* structure `RichStructure` that also contains the fields `(n+1)st_field`, ..., `rth_field`. We can either

- forget that we had `PoorStructure` and declare

```
structure RichStructure where
  firstfield : firstType
  secondfield : secondType
  ...
  rth_field : rth_Type
```

- declare that `RichStructure` **extends** `PoorStructure` inheriting terms from the latter:

```
structure RichStructure extends PoorStructure where
  (n+1)st_field : (n+1)st_Type
  ...
  rth_field : rth_Type
```

+++ In details (*probably skipped during lecture*)

- The process can be iterated, yielding a structure extending several ones:

`VeryRichStructure` extends `Structure1`, `Structure2`, `Structure3` where
...

- If the parent structures have overlapping field names, then all overlapping field names must have the **same type**.
 - If the overlapping fields have different default values, then the default value from the **last** parent structure that includes the field is used. New default values in the child (= richer) structure take precedence over default values from the parent (= poorer) structures.
 - The `with` (and `___`) syntax are able to "read through" the extension of structures.
- +++



Classes and Class Inference

+++ Some automation we've just witnessed

1. Lean was able to "automatically" decide to use `1` and `*` for `G : Group` or `G : CommGroup`, and to use `0` and `+` for `A : AddGroup` or `A : AddCommGroup`.
2. If we inspect `mul_assoc` we see

```
mul_assoc {G : Type*} [Semigroup G] (a b c : G) : a * b * c = a * (b * c)
```

but we used it for a group: Lean understood that every group is a semigroup.

3. The use of `extend` to define `Group`, yielding an "enriched" `DivInvMonoid`.
4. Some redundancy in the definition of `Group` (of `Monoid`, actually) concerning `npow : ℕ → M → M`.

+++

Most of the above points are related to *classes* and *class type inference*.

Classes are special structures, for which certain terms are stored in a database. They enable **class type inference**, constructing a term of a certain class given a term of a "parent" one.

The idea is that each `class` type contains a *preferred* or a *canonical* term, declared using the keyword `instance` rather than `def`; and this term has been registered in the database to be accessible whenever needed.

Warning: often, Classes have parameters, so if ``G`` and ``H`` are types, ``Group G`` and ``Group H`` are different types!

To introduce a class assumption in a lemma (*i. e.* to add a term to the local context), in a definition, or in a new class, we use `[` and `]` and typically *we don't name it*: because this should be useless.

To check what is the canonical term of a certain class type, use the command `#synth`, and to recall it use `inferInstance` (in term mode) or `infer_instance` (in tactic mode).

⌘

More about groups

+++ Interlude: `Mathlib`

Main take-home message: it is huge!

You can access it on its [gitHub repo](#) or, **much better**, through its [documentation website](#). It changes very rapidly (>10 times a day), so I created a "frozen" [documentation](#) website with the Mathlib version used in this course.

As a rule of thumb:

- if something is below PhD level it is *probably* in Mathlib. Unless it is not.
- Generality is normally overwhelming, so if you don't find something you're probably looking in the wrong place.
- There is a [naming convention](#) for theorems (terms of `Prop`-valued types), for objects (terms in `Type*`), for properties (terms in `Prop`); together with plenty of exceptions and room for headache. Try to develop your feeling and ask for help.

+++

Subgroups

The definition of subgroups is slightly different from that of a group, it relies on `Sets`:

```
structure Subgroup (G : Type*) [Group G] extends Submonoid G : Type* where
  carrier : Set G
  mul_mem' {a b : G} : a ∈ self.carrier → b ∈ self.carrier → a * b ∈
self.carrier
  one_mem' : 1 ∈ self.carrier
  inv_mem' {x : G} : x ∈ self.carrier → x⁻¹ ∈ self.carrier
```

We'll discuss what "sets" are in the next class, but for now just record that given any type `G : Type*`, and any set `S : Set G`, we obtain for every `g : G` the type (of kind `Prop`) `g ∈ S`, that can be either inhabited (the proposition is true) or empty (the proposition is false).

Exactly as we've discussed for monoids, that of "being a subgroup" is not a `Prop`-like notion: rather, to each group `G` we attach a new *type* `Subgroup G` whose terms represent the different subgroups of `G`, each seen as an underlying set and a collection of proofs that the set is multiplicatively closed (a "mixin").

Finally, the type `Subgroup G` is ordered, and `{1} : Subgroup G` is actually $\perp$ (the bottom element, written `\bot`) whereas `G : Subgroup G` is $\top$ (the top element, written `\top`).

Homomorphisms

Given *monoids* M and N , a *monoid homomorphism* is a function $f : M \rightarrow N$ that respects the operations and the units. There could be (at least) two ways to define this:

1. Declare the property $\text{MonHom} : (M \rightarrow N) \rightarrow \text{Prop}$ as

```
def MonHom : (M → N) → Prop := f ↦ ( ∀ a b, f (a * b) = (f a) * (f b)
) ∧ (f 1 = 1)
```

and let MonoidHom be the subtype

```
MonoidHom = {f : M → N // MonHom f}
```

This would mean that a monoid homomorphism is a pair $\langle f, hf : \text{MonHom } f \rangle$.

2. Define a new type $\text{MonoidHom } M \ N$, as a structure

```
structure MonoidHom M N where
| toFun : M → N
| map_mul : ∀ a b, toFun (a * b) = (toFun a) * (toFun b)
| map_one : toFun 1 = 1
```

so that terms of $\text{MonoidHom } M \ N$ would be *triples* $\langle f, \text{map_mul } f, \text{map_one } f \rangle$.

These approaches are not *very* different, the problem with the first is that to access the proofs one has to destructure hf to $hf.1$ and $hf.2$. Imagine if there were 20 properties...

- A **group homomorphism** is just a monoid homomorphism: the property $f(x^{-1}) = (f x)^{-1}$ can be proven. But this relies on class type inference...

+++ Why does it rely on class type inference?

Because if we only define monoid homomorphisms, then when G and H are groups and we write $f : G \rightarrow^* H$ Lean would complain that G and H are not monoids. By class type inference, it understands that they are *also* monoids.

+++

- **Take-home message:** homomorphisms between algebraic structures are structures on their own, "bundling" together the underline function and all its properties.

It will be another story for continuous/differentiable/smooth functions...

Quotients

To define quotients, we clearly need equivalence relations:

```
class Setoid (α : Sort u) where
  r : α → α → Prop
  iseqv : Equivalence r
```

Observe that α is a parameter, so a term in `Setoid  $\alpha$` is an equivalence relation on α . This comes with a notation $a \approx b$ (typed as `\~~`) meaning that $r\ a\ b$, i. e. $a, b : \alpha$ are equivalent.

+++ Why a class instead of a structure? Isn't this an obsession?

It would not change much, and in many *explicit* occurrences terms of `Setoid  $\alpha$` are actually named and in parenthesis, as in `(s : Setoid  $\alpha$ )`. But we want automation, for example in order to find the *canonical* equivalence relation on G / H under the assumption that H is normal, and not having Lean asking us to choose one relation each time.

+++

Given `s : Setoid  $\alpha$` we can construct `Quotient s`: it is a type whose terms correspond to equivalence classes of `s`, yet **I cannot define it because it is a Lean primitive**.

But the important things are not definitions, **it is the API**:

- `Quotient.mk s :  $\alpha \rightarrow$  Quotient s` is the quotient map (and `Quotient.mk _ x` is denoted $\llbracket x \rrbracket$, with `\[[` and `\]]`);
- `Quotient.out : Quotient s  $\rightarrow$   $\alpha$` sends $x : \text{Quotient } s$ to a lift of x , **using the axiom of choice**;
- `Quotient.lift`: given a term $f : \alpha \rightarrow \beta$ satisfying $a \approx b \rightarrow f\ a = f\ b$, the term `Quotient.lift f : Quotient s  $\rightarrow$   $\beta$` satisfies $(\text{Quotient.lift } f) \circ \text{Quotient.mk } s = f$;

+++ And two more esoteric ones:

- `Quotient.rep [Encodable  $\alpha$ ] : Quotient s  $\rightarrow$   $\alpha$` without using the axiom of choice, provided that α is **encodable**, i. e. it is endowed with a (chosen) bijection into a subset of $\mathbb{N}$ (we also need `s.r` to be decidable: for all a, b , either $a \approx b$ or $\neg a \approx b$).
- `axiom Quotient.sound {a b :  $\alpha$ } : a  $\approx$  b  $\rightarrow$   $\llbracket a \rrbracket = \llbracket b \rrbracket$` : it is (almost) an axiom.

+++

In the special case of `{G : Type*} [Group G] (H : Subgroup G)` there are two relations: $x\ y \mapsto x * y^{-1} \in H$ and $x\ y \mapsto x^{-1} * y \in H$ (can you understand their *signature*?), giving rise to

- the corresponding `Quotient (QuotientGroup.leftRel H) : Type*`, denoted G / H (where $/$ is `\/,` or `\quot` and not `/`);
- the corresponding `Quotient (QuotientGroup.rightRel H) : Type*`;
- the corresponding bare function `Quotient.mk (QuotientGroup.leftRel H) : G  $\rightarrow$  G / H`;
- the *much more interesting* `Quotient.mk' _ : [_ : H.Normal]  $\rightarrow$  G  $\rightarrow^*$  G / H`.

⌘

Rings

From the theoretical point of view, all algebraic structures look alike: so I'll present a (slightly long) summary of what we need about rings, and then there will be plenty of examples.

Definition and basic tactics

As for groups, the way to say that R is a ring is to type

```
(R : Type*) [Ring R]
```

The library is particularly rich insofar as *commutative* rings are concerned, and we're going to stick to those in our course.

- The tactic `ring` solves claim about basic relations in commutative rings, namely those that hold in *free* rings.
- The tactic `grind` is much more powerful (but it often calls the axiom of choice for no good reason: we won't care): beyond what `ring` can do, it also treats inequalities, logical connectives, local assumptions, etc.

Given what we know about groups and monoids, we can expect a commutative ring to have several "weaker" structures: typically these can be accessed through a `.toWeakStructure` projection.

Morphisms, Ideals and Quotients.

- Morphisms work as for groups: they are functions respecting both structures on a ring, that of a multiplicative monoid and of an additive group: so, they're respecting both monoid structures, hence the notation $R \rightarrow^{+*} S$ for a ring homomorphism. Of course, $\simeq^{+*}$ denotes ring isomorphism, so $R \simeq^{+*} S$ is the type of all ring homomorphisms from R to S .
- In the usual spirit of working in the greatest possible generality, $I : \text{Ideal } R$ means $I : \text{Submodule } R R$.

Concretely, a term $I : \text{Ideal } R$ is such that

```
| I.carrier : Set R
| I.zero_mem : (0 : R) ∈ I
| I.add_mem (x y : R) : x ∈ I → y ∈ I → x + y ∈ I
| I.smul_mem (x y : R) : x ∈ I → x • y ∈ I
```

The `smul_mem` field is part of the definition, but it is sometimes handier to use either of

```
I.mul_mem_left (a b : R) : b ∈ I → a * b ∈ I
```

or

```
I.mul_mem_right (a b : R) : a ∈ I → a * b ∈ I
```

As for subgroups, the type `Ideal R` is ordered, with $\perp = \{0\} : \text{Ideal } R$ and $\top = R : \text{Ideal } R$.

- The discussion about quotient rings is analogous to the one for quotient groups, through the *same* setoid structure (called `Submodule.quotientRel`)

```
@[to_additive]
def leftRel (s : Subgroup α) : Setoid α := MulAction.orbitRel s.op α
```

where `to_additive` fires, becoming `fun x y ↦ x - y ∈ I.toAddSubgroup`.

As for groups, it is often better to replace `Quotient.mk (Submodule.quotientRel I)` by `Ideal.Quotient.mk I : R →+* R / I`.

⌘